

QUEUE IMPLEMENTATIONS IN C

Queue Using Arrays

```
#include <stdio.h>

#define MAX 100

int queue[MAX];
int front = -1;
int rear = -1;

void enqueue(int x) {
    if (rear == MAX - 1)
        printf("Queue Overflow\n");
    else {
        if (front == -1)
            front = 0;

        queue[++rear] = x;
    }
}

void dequeue() {
    if (front == -1 || front > rear)
        printf("Queue Underflow\n");
    else {
        printf("Deleted element: %d\n", queue[front++]);

        if (front > rear)
            front = rear = -1;
    }
}

void peek() {
    if (front == -1)
        printf("Queue is empty\n");
    else
        printf("Front element: %d\n", queue[front]);
}

void display() {
    if (front == -1)
        printf("Queue is empty\n");
    else {
        printf("Queue elements:\n");
```

```
        for (int i = front; i <= rear; i++)
            printf("%d\n", queue[i]);
    }
}

int main() {
    int choice, x;

    while (1) {
        printf("\n1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit\n");
        scanf("%d", &choice);

        switch (choice) {

            case 1:
                printf("Enter value: ");
                scanf("%d", &x);
                enqueue(x);
                break;

            case 2:
                dequeue();
                break;

            case 3:
                peek();
                break;

            case 4:
                display();
                break;

            case 5:
                return 0;

            default:
                printf("Invalid choice\n");
        }
    }
}
```

Queue Using Array with Structure

```
#include <stdio.h>

#define MAX 100

struct Queue {
    int arr[MAX];
    int front;
    int rear;
};

void init(struct Queue *q) {
    q->front = -1;
    q->rear = -1;
}

void enqueue(struct Queue *q, int x) {
    if (q->rear == MAX - 1)
        printf("Queue Overflow\n");
    else {
        if (q->front == -1)
            q->front = 0;

        q->arr[++(q->rear)] = x;
    }
}

void dequeue(struct Queue *q) {
    if (q->front == -1 || q->front > q->rear)
        printf("Queue Underflow\n");
    else {
        printf("Deleted element: %d\n", q->arr[(q->front)++]);

        if (q->front > q->rear)
            q->front = q->rear = -1;
    }
}

void peek(struct Queue *q) {
    if (q->front == -1)
        printf("Queue is empty\n");
    else
        printf("Front element: %d\n", q->arr[q->front]);
}

void display(struct Queue *q) {
```

```
    if (q->front == -1)
        printf("Queue is empty\n");
    else {
        printf("Queue elements:\n");

        for (int i = q->front; i <= q->rear; i++)
            printf("%d\n", q->arr[i]);
    }
}

int main() {
    struct Queue q;
    init(&q);

    int choice, x;

    while (1) {
        printf("\n1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit\n");
        scanf("%d", &choice);

        switch (choice) {

            case 1:
                printf("Enter value: ");
                scanf("%d", &x);
                enqueue(&q, x);
                break;

            case 2:
                dequeue(&q);
                break;

            case 3:
                peek(&q);
                break;

            case 4:
                display(&q);
                break;

            case 5:
                return 0;

            default:
                printf("Invalid choice\n");
        }
    }
}
```

Queue Using Linked List

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node *front = NULL, *rear = NULL;

void enqueue(int x) {
    struct Node* newNode =
        (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Queue Overflow\n");
    } else {
        newNode->data = x;
        newNode->next = NULL;

        if (rear == NULL) {
            front = rear = newNode;
        } else {
            rear->next = newNode;
            rear = newNode;
        }
    }
}

void dequeue() {
    if (front == NULL) {
        printf("Queue Underflow\n");
    } else {
        struct Node* temp = front;

        printf("Deleted element: %d\n", front->data);

        front = front->next;

        if (front == NULL)
            rear = NULL;

        free(temp);
    }
}
```

```
void peek() {
    if (front == NULL)
        printf("Queue is empty\n");
    else
        printf("Front element: %d\n", front->data);
}

void display() {
    if (front == NULL)
        printf("Queue is empty\n");
    else {
        struct Node* temp = front;

        printf("Queue elements:\n");

        while (temp != NULL) {
            printf("%d\n", temp->data);
            temp = temp->next;
        }
    }
}

int main() {
    int choice, x;

    while (1) {
        printf("\n1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit\n");
        scanf("%d", &choice);

        switch (choice) {

            case 1:
                printf("Enter value: ");
                scanf("%d", &x);
                enqueue(x);
                break;

            case 2:
                dequeue();
                break;

            case 3:
                peek();
                break;

            case 4:
                display();
                break;
        }
    }
}
```

```
        case 5:
            return 0;

        default:
            printf("Invalid choice\n");
    }
}
```